

Problem 294: Looking a Bit Rusty

Difficulty: Medium

Author: Javier Jimenez, Marietta, Georgia, United States

Originally Published: Code Quest 2026

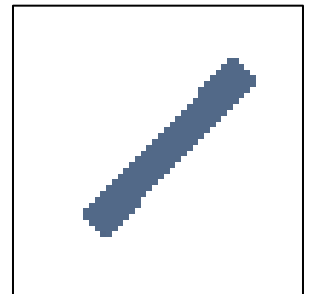
Problem Background

Lockheed Martin manufactures around 150 F-35 fighter aircraft each year for the US Department of Defense. In order to ensure that those manufacturing goals are met, we have to ensure that all the parts we use are of the highest quality and free of defects. To streamline this process and increase production goals further, Lockheed Martin Aeronautics is planning to use a new artificial-intelligence system to scan parts prior to their installation to check for signs of rust or other damage. They'd like you to work on a prototype for this system that can serve as a proof-of-concept.

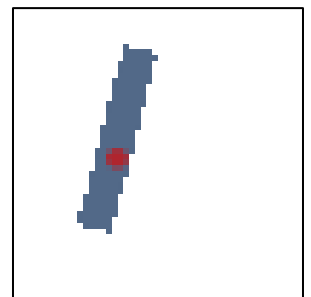
Problem Description

Aircraft parts will be photographed by a series of cameras under consistent lighting, then provided to your program in X PixMap, or XPM, format. XPM files are text-based image files, that use ASCII characters to represent individual pixels in an image. The format is unusually human-readable for an image file, and is actually designed to be interpreted as an array of strings in a C program. The files you'll be given are structured as follows:

- The first line contains a comment identifying the file as an XPM file (this is irrelevant to this problem):
`/* XPM */`
- The second line declares the string array in C format (also irrelevant for our purposes):
`static char * file_name_here_xpm[] = {`
- The third line is the first one to contain any useful information; it contains a string with four integers, separated by spaces:
 - The width of the image, in pixels
 - The height of the image, in pixels
 - The number of colors defined in the image
 - The number of characters used to define each pixel (for this problem, this value will always be 1)
- The next several lines – the exact number determined by the third value in the previous string – will contain strings that define the colors used in the image. Each color is defined as follows:
 - An ASCII character (including spaces!) used to represent a pixel in the image
 - A tab character



Sample Input 1, without rust



Sample Input 2, rusty

- A lowercase letter indicating the type of the color; for this problem, this will always be 'c', for "color"
- A space
- A hexadecimal RGB color string, such as #FFFFFF for white or #003478 for the blue used in Lockheed Martin's star logo. Alternatively, this may be "None" to represent a transparent pixel that has no color.
- The remaining lines contain strings representing the image itself. The number of strings will be equal to the image's height, and the length of each string equal to its width. Each character represents a single pixel; the color of that pixel is determined using the color mapping provided above.

All of the images you'll be examining will be made of the same metallic material, so should be a consistent color: specifically, #526988. You may see slightly different colors, particularly around the edges of a part, due to the way light reflects off a 3-dimensional part being rendered in a 2-dimensional image. However, if you see a contiguous square of pixels (at least two pixels in each dimension) consisting entirely of colors other than what we expect, that may indicate that the part is rusted, corroded, or otherwise damaged. It should be flagged for a manual inspection before it's used on an aircraft. Transparent pixels will have the "None" color and should not be considered in a part's evaluation.

Sample Input

The first line of your program's input, **received from the standard input channel**, will contain a positive integer representing the number of test cases. Each test case will include a single image in XPM format; while this format is generally outlined above, to review:

- The first two lines of each test case contain no relevant data and may be ignored
- The third line of each test case will contain a string (wrapped with double quotes, and followed by a comma) containing the following positive integer values separated by spaces:
 - W, the width of the image in pixels
 - H, the height of the image in pixels
 - C, the number of colors defined for potential use in the image
 - P, the number of characters representing each pixel. This value will always be 1.
- C lines containing color definitions. Not all colors defined here will necessarily be used within an image. These are also defined as strings (wrapped with double quotes, and followed by a comma), using the following format:
 - A single printable ASCII character, which is the character that will be used to represent pixels in this color. This character will not be a tab, double quote, or backslash, but may be any other ASCII character appearing in the table in the Reference Materials.
 - A tab (\t), a lowercase c, and a space
 - A hexadecimal color code, prefaced with a # character; or the word "None"

- H lines, each containing a string of length W wrapped in double quotes and followed by a comma (except for the last line, which is followed by a closing curly bracket and semicolon). These strings represent the pixels in the image, and will contain characters listed in the color definition section above.

Due to the length of the sample input, it is not replicated here; please download the sample input from the contest website's "Problemset" tab.

Sample Output

For each test case, your program must print a single line of text, as follows:

- The word "PASS" if the image does not contain any visible defects
- The word "FAIL" if the image shows visible defects in the part, as described above

PASS

FAIL